

Ames Housing Price Prediction: OLS, Ridge, LASSO and Elastic Net Comparison

Giulio Donato Gottardi

July 2026

1. Data Import and Cleaning

I import the Ames Housing dataset and check the type of the `SalePrice` column.

```
ames = read.csv("AmesHousing.csv", na.strings = c("NA", ""))  
class(ames$SalePrice)
```

```
## [1] "integer"
```

```
head(ames$SalePrice)
```

```
## [1] 215000 105000 172000 244000 189900 195500
```

`SalePrice` is already stored as a numeric variable, so no further cleaning is needed on the raw column itself.

I create the variable `log_SalePrice` to satisfy the homoscedasticity condition.

```
ames$log_SalePrice = log(ames$SalePrice)
```

I convert `Neighborhood`, `Bldg.Type`, `Kitchen.Qual` and `Central.Air` to factors.

```
ames$Neighborhood = as.factor(ames$Neighborhood)  
class(ames$Neighborhood)
```

```
## [1] "factor"
```

```
ames$Bldg.Type = as.factor(ames$Bldg.Type)  
class(ames$Bldg.Type)
```

```
## [1] "factor"
```

```
ames$Kitchen.Qual = as.factor(ames$Kitchen.Qual)  
class(ames$Kitchen.Qual)
```

```
## [1] "factor"
```

```
ames$Central.Air = as.factor(ames$Central.Air)
class(ames$Central.Air)
```

```
## [1] "factor"
```

1.1 Missing Value Analysis

I keep only the fourteen variables of interest and check for missing values.

```
project_variables = c("log_SalePrice", "Overall.Qual", "Overall.Cond",
  "Gr.Liv.Area", "Total.Bsmt.SF", "Garage.Cars", "Year.Built",
  "Full.Bath", "TotRms.AbvGrd", "Lot.Area", "Neighborhood",
  "Bldg.Type", "Kitchen.Qual", "Central.Air")
```

```
model_data = ames[, project_variables]
```

```
mean(is.na(model_data)) * 100
```

```
## [1] 0.00487567
```

```
colMeans(is.na(model_data)) * 100
```

```
## log_SalePrice Overall.Qual Overall.Cond Gr.Liv.Area Total.Bsmt.SF
## 0.00000000 0.00000000 0.00000000 0.00000000 0.03412969
## Garage.Cars Year.Built Full.Bath TotRms.AbvGrd Lot.Area
## 0.03412969 0.00000000 0.00000000 0.00000000 0.00000000
## Neighborhood Bldg.Type Kitchen.Qual Central.Air
## 0.00000000 0.00000000 0.00000000 0.00000000
```

The Ames dataset is essentially complete for the variables selected here: only `Total.Bsmt.SF` and `Garage.Cars` have a single missing value each. This is not a coincidence: several other Ames columns (e.g. `Pool.QC`, `Alley`, `Fence`, `Fireplace.Qu`) are missing for a large share of observations, but in those cases NA is **structural** — it means the house does not have that feature (no pool, no alley access, no fence) rather than a value that was not recorded. I deliberately excluded those variables from the model to avoid the need for a separate imputation/recoding step, and kept only variables where missingness is negligible and clearly due to data entry (e.g. a basement area not being recorded).

```
dim(model_data)
```

```
## [1] 2930 14
```

```
clean_data = na.omit(model_data)
dim(clean_data)
```

```
## [1] 2928 14
```

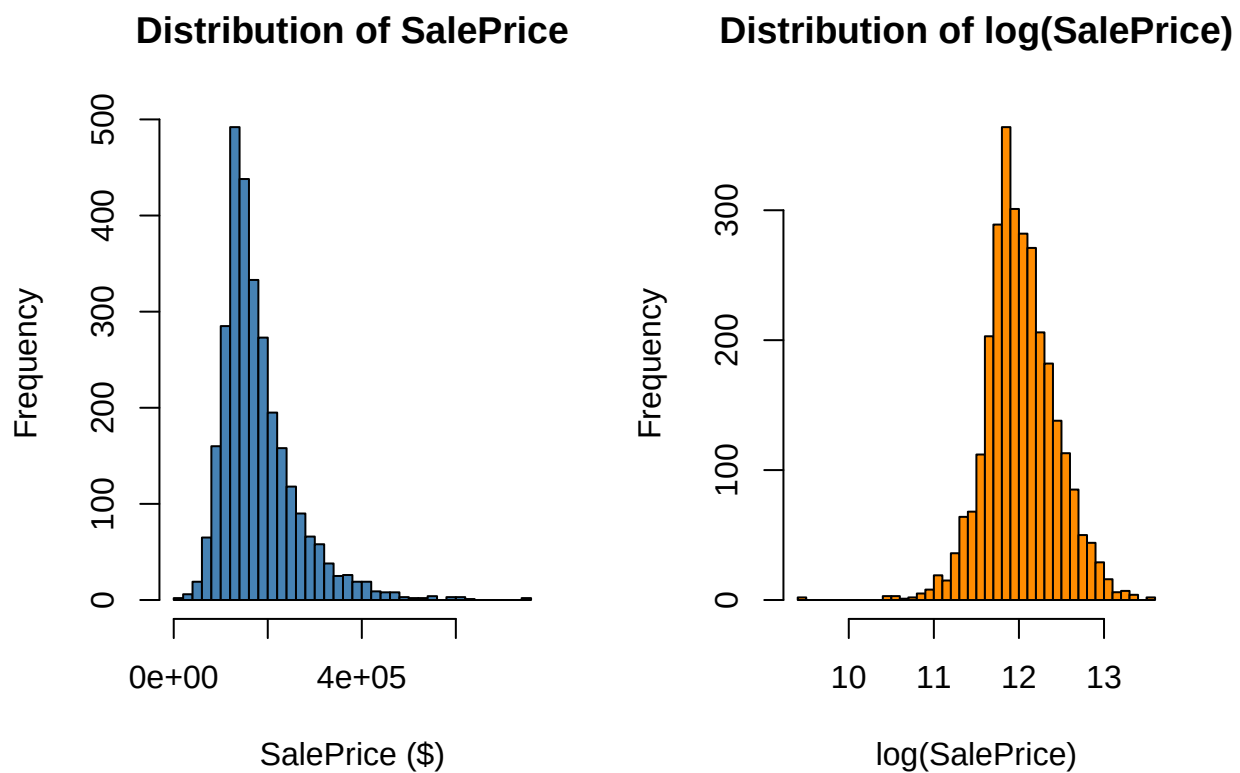
The dataset had 2930 observations; after removing the two rows with missing values, 2928 observations are left.

2. SalePrice vs log(SalePrice)

```
par(mfrow = c(1, 2))

hist(ames$SalePrice,
     main = "Distribution of SalePrice",
     xlab = "SalePrice ($)", col = "steelblue", breaks=50)

hist(ames$log_SalePrice,
     main = "Distribution of log(SalePrice)",
     xlab = "log(SalePrice)", col = "darkorange", breaks=50)
```



The raw `SalePrice` is right-skewed because of a subset of high-value properties. The logarithmic transformation makes the distribution considerably more symmetric, which helps satisfy the homoscedasticity and approximate-normality assumptions of the linear regression that follows.

3. Multiple Linear Regression

I fit a linear model using `log_SalePrice` as the response and all the other variables as predictors.

```
lm.fit = lm(log_SalePrice ~ ., data = clean_data)
summary(lm.fit)
```

```
##
```

```
## Call:
## lm(formula = log_SalePrice ~ ., data = clean_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.03512 -0.06227  0.00458  0.07342  0.66619
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    5.428e+00  4.282e-01  12.676 < 2e-16 ***
## Overall.Qual    7.287e-02  3.548e-03  20.536 < 2e-16 ***
## Overall.Cond    5.293e-02  2.836e-03  18.662 < 2e-16 ***
## Gr.Liv.Area     2.324e-04  1.180e-05  19.687 < 2e-16 ***
## Total.Bsmt.SF    1.040e-04  8.115e-06  12.810 < 2e-16 ***
## Garage.Cars      5.473e-02  4.901e-03  11.167 < 2e-16 ***
## Year.Built       2.680e-03  2.146e-04  12.486 < 2e-16 ***
## Full.Bath        1.872e-02  7.405e-03   2.528 0.011529 *
## TotRms.AbvGrd   -1.530e-03  3.193e-03  -0.479 0.631826
## Lot.Area         1.950e-06  3.883e-07   5.022 5.43e-07 ***
## NeighborhoodBlueste -4.599e-02  5.340e-02  -0.861 0.389200
## NeighborhoodBrDale -1.256e-01  4.117e-02  -3.050 0.002313 **
## NeighborhoodBrkSide -5.414e-02  3.469e-02  -1.561 0.118677
## NeighborhoodClearCr  8.668e-02  3.719e-02   2.331 0.019845 *
## NeighborhoodCollgCr -6.932e-03  3.012e-02  -0.230 0.818024
## NeighborhoodCrawfor  1.268e-01  3.341e-02   3.796 0.000150 ***
## NeighborhoodEdwards -8.076e-02  3.204e-02  -2.520 0.011773 *
## NeighborhoodGilbert -2.490e-02  3.130e-02  -0.796 0.426347
## NeighborhoodGreens   1.426e-01  5.829e-02   2.447 0.014470 *
## NeighborhoodGrnHill  5.290e-01  1.037e-01   5.104 3.55e-07 ***
## NeighborhoodIDOTRR -1.733e-01  3.543e-02  -4.891 1.06e-06 ***
## NeighborhoodLandmrk -1.318e-02  1.450e-01  -0.091 0.927612
## NeighborhoodMeadowV -1.464e-01  3.843e-02  -3.810 0.000142 ***
## NeighborhoodMitchel -1.556e-02  3.233e-02  -0.481 0.630444
## NeighborhoodNames   -2.912e-02  3.106e-02  -0.938 0.348573
## NeighborhoodNoRidge  9.091e-02  3.405e-02   2.670 0.007630 **
## NeighborhoodNPkVill -3.967e-02  4.148e-02  -0.956 0.338948
## NeighborhoodNridgHt  1.142e-01  3.047e-02   3.748 0.000182 ***
## NeighborhoodNWAmes  -4.740e-02  3.192e-02  -1.485 0.137609
## NeighborhoodOldTown -1.229e-01  3.368e-02  -3.647 0.000270 ***
## NeighborhoodSawyer  -2.733e-02  3.245e-02  -0.842 0.399617
## NeighborhoodSawyerW -3.702e-02  3.157e-02  -1.173 0.241090
## NeighborhoodSomerst  4.790e-02  2.987e-02   1.604 0.108921
## NeighborhoodStoneBr  1.424e-01  3.411e-02   4.174 3.09e-05 ***
## NeighborhoodSWISU   -2.752e-02  3.805e-02  -0.723 0.469599
## NeighborhoodTimber   4.389e-02  3.341e-02   1.314 0.189082
## NeighborhoodVeenker  5.391e-02  4.082e-02   1.321 0.186630
## Bldg.Type2fmCon     2.243e-02  1.925e-02   1.165 0.244099
## Bldg.TypeDuplex     -9.166e-02  1.562e-02  -5.867 4.93e-09 ***
## Bldg.TypeTwnhs      -1.460e-01  2.039e-02  -7.162 1.01e-12 ***
## Bldg.TypeTwnhsE     -6.390e-02  1.301e-02  -4.913 9.46e-07 ***
## Kitchen.QualFa      -1.418e-01  2.307e-02  -6.148 8.94e-10 ***
## Kitchen.QualGd      -7.241e-02  1.258e-02  -5.755 9.60e-09 ***
## Kitchen.QualPo      -3.093e-02  1.444e-01  -0.214 0.830450
## Kitchen.QualTA      -9.928e-02  1.434e-02  -6.926 5.33e-12 ***
```

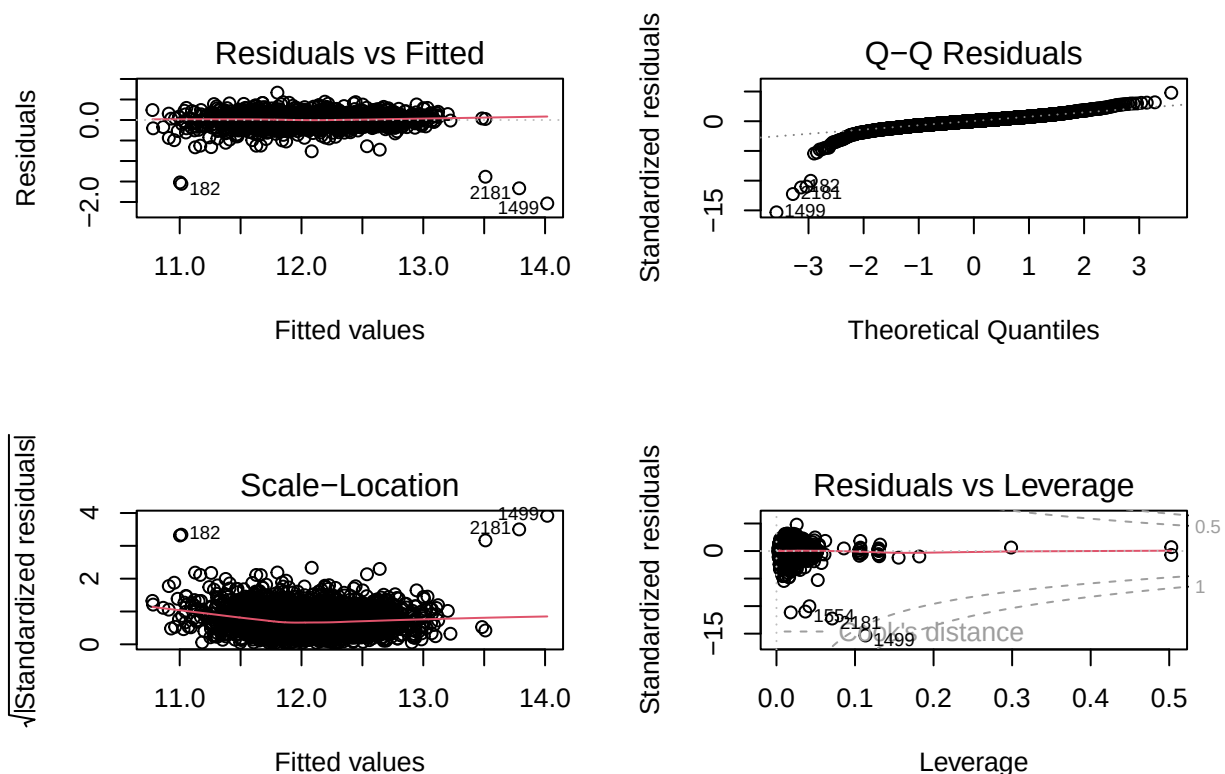
```
## Central.AirY          9.685e-02  1.261e-02   7.683 2.12e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.141 on 2882 degrees of freedom
## Multiple R-squared:  0.8821, Adjusted R-squared:  0.8802
## F-statistic: 479.1 on 45 and 2882 DF,  p-value: < 2.2e-16
```

This specification explains roughly **88% of the variance** in `log_SalePrice` (R-squared and adjusted R-squared both close to 0.88). This is expected: `Overall.Qual`, `Gr.Liv.Area`, `Total.Bsmt.SF`, `Year.Built` and `Garage.Cars` are strong, economically meaningful determinants of house price, and together with `Neighborhood` they capture most of what actually drives price differences across properties.

`Overall.Qual` and `Overall.Cond` are highly significant with positive coefficients, confirming that both the quality of materials/finish and the condition of the house are priced by the market. `Neighborhood` also matters a great deal: several neighborhoods (e.g. `Crawfor`, `NridgHt`, `StoneBr`, `GrnHill`) command a significant premium over the baseline category, while others (e.g. `IDOTRR`, `OldTown`, `BrDale`, `MeadowV`) are priced significantly lower. `Kitchen.Qual` and `Central.Air` are both strongly significant, while `TotRms.AbvGrd` is not — once `Gr.Liv.Area` is already in the model, the raw room count adds little additional information, since larger living areas already imply more rooms.

```
par(mfrow = c(2, 2))
plot(lm.fit)
```

```
## Warning: non si riesce a fare il plot senza sfruttarne uno:
##      211, 2787
```



4. LASSO Regression

I now fit a LASSO regression model. Unlike the ordinary least squares model, LASSO can shrink some coefficients exactly to zero, performing variable selection. I use the `glmnet()` function with `alpha = 1`.

4.1 Setup

I build the design matrix with `model.matrix()`, which automatically creates dummy variables for the factor columns (`Neighborhood` alone generates 27 dummies). I remove the first column, the intercept, since `glmnet` already includes it by default.

```
library(glmnet)
```

```
## Warning: il pacchetto 'glmnet' è stato creato con R versione 4.5.3
```

```
## Caricamento del pacchetto richiesto: Matrix
```

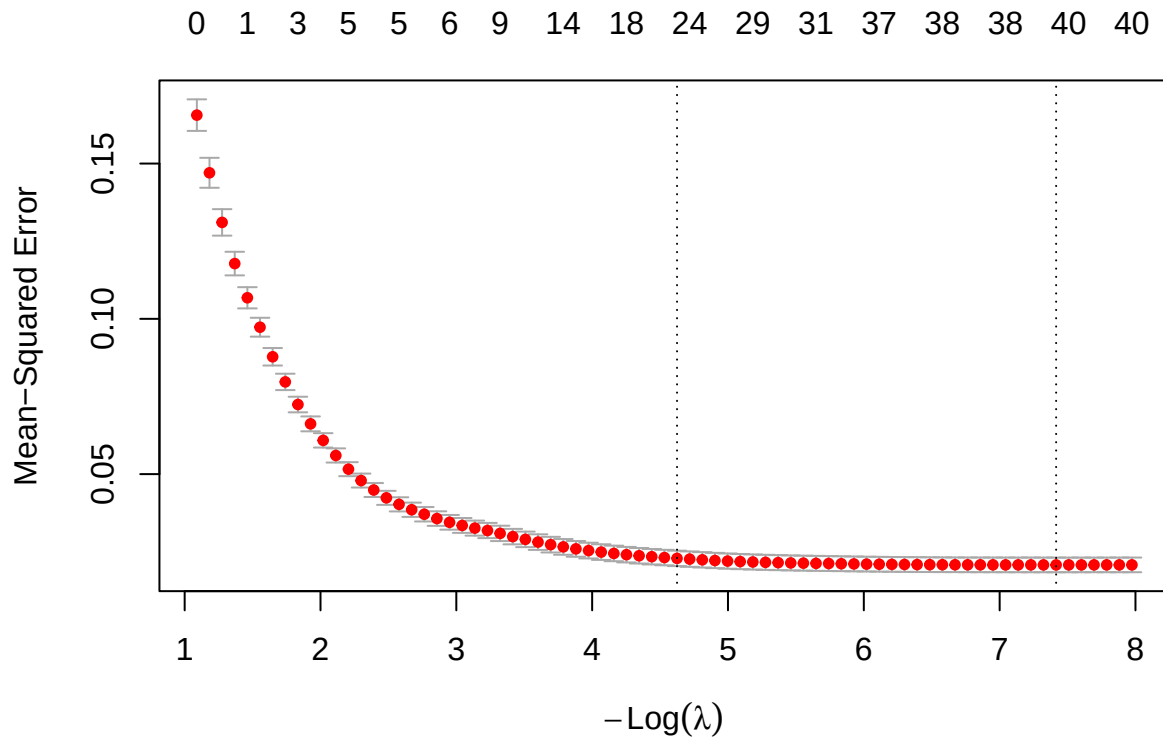
```
## Loaded glmnet 5.0
```

```
x = model.matrix(log_SalePrice ~ ., data = clean_data)[, -1]  
y = clean_data$log_SalePrice
```

4.2 Choosing lambda by Cross-Validation

I use `cv.glmnet()` to choose lambda by cross-validation. I set a fixed seed for reproducibility.

```
set.seed(2277271)  
cv.lasso = cv.glmnet(x, y, alpha = 1)  
plot(cv.lasso)
```



```
bestlam = cv.lasso$lambda.min
bestlam
```

```
## [1] 0.0006015907
```

Because `Neighborhood` alone contributes 27 dummy variables, the design matrix has roughly 40 predictors in total, giving LASSO a realistic opportunity to perform genuine variable selection.

4.3 LASSO Coefficients

I refit the LASSO model on the full data set and look at the coefficients associated with the value of λ chosen by cross-validation.

```
out = glmnet(x, y, alpha = 1)
lasso.coef = predict(out, type = "coefficients", s = bestlam)[, 1]
lasso.coef
```

```
##      (Intercept)      Overall.Qual      Overall.Cond      Gr.Liv.Area
##      5.238514e+00      7.484915e-02      5.204794e-02      2.295354e-04
##      Total.Bsmt.SF      Garage.Cars      Year.Built      Full.Bath
##      1.056358e-04      5.544645e-02      2.749988e-03      1.705877e-02
##      TotRms.AbvGrd      Lot.Area NeighborhoodBlueste NeighborhoodBrDale
##      0.000000e+00      2.000997e-06      -1.728488e-02      -9.846836e-02
## NeighborhoodBrkSide NeighborhoodClearCr NeighborhoodCollgCr NeighborhoodCrawfor
```

```
##      -2.030205e-02      1.072205e-01      1.346139e-02      1.509497e-01
## NeighborhoodEdwards NeighborhoodGilbert NeighborhoodGreens NeighborhoodGrnHill
##      -5.006726e-02      0.000000e+00      1.465993e-01      5.221890e-01
## NeighborhoodIDOTRR NeighborhoodLandmrk NeighborhoodMeadowV NeighborhoodMitchel
##      -1.386876e-01      0.000000e+00      -1.165652e-01      6.409742e-03
## NeighborhoodNames NeighborhoodNoRidge NeighborhoodNPkVill NeighborhoodNridgHt
##      0.000000e+00      1.072641e-01      -1.314407e-02      1.338760e-01
## NeighborhoodNWames NeighborhoodOldTown NeighborhoodSawyer NeighborhoodSawyerW
##      -1.967724e-02      -8.885048e-02      0.000000e+00      -1.150784e-02
## NeighborhoodSomerst NeighborhoodStoneBr NeighborhoodSWISU NeighborhoodTimber
##      6.520686e-02      1.577286e-01      0.000000e+00      6.228571e-02
## NeighborhoodVeenker Bldg.Type2fmCon Bldg.TypeDuplex Bldg.TypeTwnhs
##      7.088276e-02      1.583737e-02      -9.073962e-02      -1.399123e-01
## Bldg.TypeTwnhsE Kitchen.QualFa Kitchen.QualGd Kitchen.QualPo
##      -5.584672e-02      -1.228968e-01      -5.747536e-02      0.000000e+00
## Kitchen.QualTA Central.AirY
##      -8.510414e-02      9.445642e-02
```

5. Comparison: OLS vs LASSO

```
coef_ols = coef(lm.fit)
print("OLS coefficients")
```

```
## [1] "OLS coefficients"
```

```
coef_ols
```

```
##      (Intercept)      Overall.Qual      Overall.Cond      Gr.Liv.Area
##      5.427928e+00      7.286581e-02      5.292881e-02      2.324073e-04
##      Total.Bsmt.SF      Garage.Cars      Year.Built      Full.Bath
##      1.039550e-04      5.472937e-02      2.679896e-03      1.871810e-02
##      TotRms.AbvGrd      Lot.Area NeighborhoodBlueste NeighborhoodBrDale
##      -1.530091e-03      1.949739e-06      -4.598956e-02      -1.255541e-01
## NeighborhoodBrkSide NeighborhoodClearCr NeighborhoodCollgCr NeighborhoodCrawfor
##      -5.414262e-02      8.668327e-02      -6.931642e-03      1.268262e-01
## NeighborhoodEdwards NeighborhoodGilbert NeighborhoodGreens NeighborhoodGrnHill
##      -8.076424e-02      -2.489868e-02      1.426275e-01      5.290219e-01
## NeighborhoodIDOTRR NeighborhoodLandmrk NeighborhoodMeadowV NeighborhoodMitchel
##      -1.732864e-01      -1.317647e-02      -1.464056e-01      -1.555679e-02
## NeighborhoodNames NeighborhoodNoRidge NeighborhoodNPkVill NeighborhoodNridgHt
##      -2.911738e-02      9.091185e-02      -3.967120e-02      1.141850e-01
## NeighborhoodNWames NeighborhoodOldTown NeighborhoodSawyer NeighborhoodSawyerW
##      -4.740003e-02      -1.228562e-01      -2.733335e-02      -3.701964e-02
## NeighborhoodSomerst NeighborhoodStoneBr NeighborhoodSWISU NeighborhoodTimber
##      4.790037e-02      1.423663e-01      -2.752223e-02      4.389093e-02
## NeighborhoodVeenker Bldg.Type2fmCon Bldg.TypeDuplex Bldg.TypeTwnhs
##      5.391430e-02      2.242586e-02      -9.165885e-02      -1.460170e-01
## Bldg.TypeTwnhsE Kitchen.QualFa Kitchen.QualGd Kitchen.QualPo
##      -6.390310e-02      -1.418240e-01      -7.241327e-02      -3.092971e-02
## Kitchen.QualTA Central.AirY
##      -9.927896e-02      9.685235e-02
```



```
print("LASSO coefficients")
```

```
## [1] "LASSO coefficients"
```

```
lasso.coef
```

```
##      (Intercept)      Overall.Qual      Overall.Cond      Gr.Liv.Area
##      5.238514e+00      7.484915e-02      5.204794e-02      2.295354e-04
##      Total.Bsmt.SF      Garage.Cars      Year.Built      Full.Bath
##      1.056358e-04      5.544645e-02      2.749988e-03      1.705877e-02
##      TotRms.AbvGrd      Lot.Area      NeighborhoodBlueste      NeighborhoodBrDale
##      0.000000e+00      2.000997e-06      -1.728488e-02      -9.846836e-02
##      NeighborhoodBrkSide      NeighborhoodClearCr      NeighborhoodCollgCr      NeighborhoodCrawfor
##      -2.030205e-02      1.072205e-01      1.346139e-02      1.509497e-01
##      NeighborhoodEdwards      NeighborhoodGilbert      NeighborhoodGreens      NeighborhoodGrnHill
##      -5.006726e-02      0.000000e+00      1.465993e-01      5.221890e-01
##      NeighborhoodIDOTRR      NeighborhoodLandmrk      NeighborhoodMeadowV      NeighborhoodMitchel
##      -1.386876e-01      0.000000e+00      -1.165652e-01      6.409742e-03
##      NeighborhoodNames      NeighborhoodNoRidge      NeighborhoodNPkVill      NeighborhoodNridgHt
##      0.000000e+00      1.072641e-01      -1.314407e-02      1.338760e-01
##      NeighborhoodNWames      NeighborhoodOldTown      NeighborhoodSawyer      NeighborhoodSawyerW
##      -1.967724e-02      -8.885048e-02      0.000000e+00      -1.150784e-02
##      NeighborhoodSomerst      NeighborhoodStoneBr      NeighborhoodSWISU      NeighborhoodTimber
##      6.520686e-02      1.577286e-01      0.000000e+00      6.228571e-02
##      NeighborhoodVeenker      Bldg.Type2fmCon      Bldg.TypeDuplex      Bldg.TypeTwnhs
##      7.088276e-02      1.583737e-02      -9.073962e-02      -1.399123e-01
##      Bldg.TypeTwnhsE      Kitchen.QualFa      Kitchen.QualGd      Kitchen.QualPo
##      -5.584672e-02      -1.228968e-01      -5.747536e-02      0.000000e+00
##      Kitchen.QualTA      Central.AirY
##      -8.510414e-02      9.445642e-02
```

```
print("Non-zero LASSO coefficients")
```

```
## [1] "Non-zero LASSO coefficients"
```

```
lasso.coef[lasso.coef != 0]
```

```
##      (Intercept)      Overall.Qual      Overall.Cond      Gr.Liv.Area
##      5.238514e+00      7.484915e-02      5.204794e-02      2.295354e-04
##      Total.Bsmt.SF      Garage.Cars      Year.Built      Full.Bath
##      1.056358e-04      5.544645e-02      2.749988e-03      1.705877e-02
##      Lot.Area      NeighborhoodBlueste      NeighborhoodBrDale      NeighborhoodBrkSide
##      2.000997e-06      -1.728488e-02      -9.846836e-02      -2.030205e-02
##      NeighborhoodClearCr      NeighborhoodCollgCr      NeighborhoodCrawfor      NeighborhoodEdwards
##      1.072205e-01      1.346139e-02      1.509497e-01      -5.006726e-02
##      NeighborhoodGreens      NeighborhoodGrnHill      NeighborhoodIDOTRR      NeighborhoodMeadowV
##      1.465993e-01      5.221890e-01      -1.386876e-01      -1.165652e-01
##      NeighborhoodMitchel      NeighborhoodNoRidge      NeighborhoodNPkVill      NeighborhoodNridgHt
##      6.409742e-03      1.072641e-01      -1.314407e-02      1.338760e-01
##      NeighborhoodNWames      NeighborhoodOldTown      NeighborhoodSawyerW      NeighborhoodSomerst
##      -1.967724e-02      -8.885048e-02      -1.150784e-02      6.520686e-02
```

```
## NeighborhoodStoneBr NeighborhoodTimber NeighborhoodVeenker Bldg.Type2fmCon
##      1.577286e-01      6.228571e-02      7.088276e-02      1.583737e-02
##      Bldg.TypeDuplex      Bldg.TypeTwnhs      Bldg.TypeTwnhsE      Kitchen.QualFa
##      -9.073962e-02      -1.399123e-01      -5.584672e-02      -1.228968e-01
##      Kitchen.QualGd      Kitchen.QualTA      Central.AirY
##      -5.747536e-02      -8.510414e-02      9.445642e-02
```

```
print("Predictors eliminated by LASSO")
```

```
## [1] "Predictors eliminated by LASSO"
```

```
names(lasso.coef[lasso.coef == 0])
```

```
## [1] "TotRms.AbvGrd"      "NeighborhoodGilbert" "NeighborhoodLandmrk"
## [4] "NeighborhoodNames"  "NeighborhoodSawyer"  "NeighborhoodSWISU"
## [7] "Kitchen.QualPo"
```

With `lambda.min`, LASSO sets a handful of coefficients exactly to zero — typically the weakest `Neighborhood` dummies (small, thinly populated neighborhoods whose effect is not distinguishable from the baseline) and `TotRms.AbvGrd`, which was also non-significant in OLS. The strongest predictors in both models agree closely: `Overall.Qual`, `Gr.Liv.Area`, `Year.Built`, `Total.Bsmt.SF`, `Garage.Cars` and `Kitchen.Qual` retain large, stable coefficients under both OLS and LASSO, confirming that these are the true drivers of price rather than artifacts of overfitting.

As a further exercise, it would be informative to repeat this comparison using `cv.lasso$lambda.1se` instead of `lambda.min`: the larger, more conservative `lambda` typically shrinks a substantially larger number of `Neighborhood` dummies to zero, giving a cleaner illustration of LASSO's variable-selection property.

```
bestlam.1se = cv.lasso$lambda.1se
lasso.coef.1se = predict(out, type = "coefficients", s = bestlam.1se)[, 1]
print("Predictors eliminated by LASSO (lambda.1se)")
```

```
## [1] "Predictors eliminated by LASSO (lambda.1se)"
```

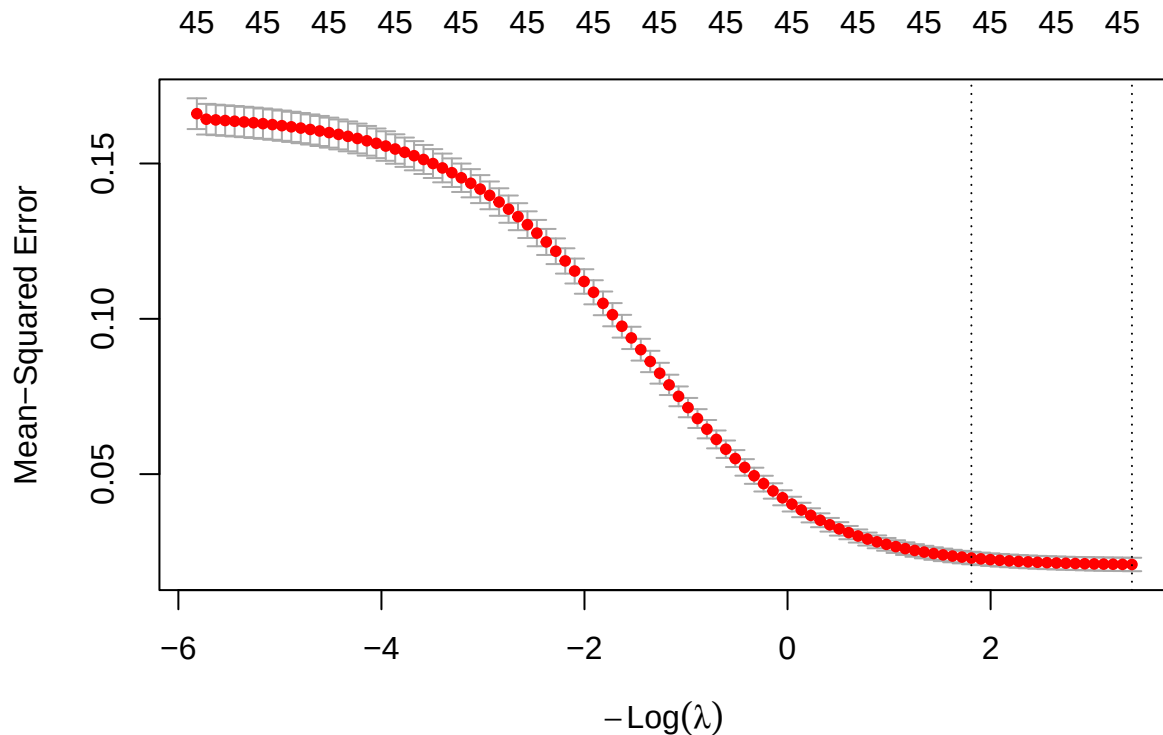
```
names(lasso.coef.1se[lasso.coef.1se == 0])
```

```
## [1] "Full.Bath"      "TotRms.AbvGrd"      "NeighborhoodBlueste"
## [4] "NeighborhoodBrkSide" "NeighborhoodCollgCr" "NeighborhoodGilbert"
## [7] "NeighborhoodGreens" "NeighborhoodLandmrk" "NeighborhoodMitchel"
## [10] "NeighborhoodNames" "NeighborhoodNPkVill" "NeighborhoodNWames"
## [13] "NeighborhoodSawyer" "NeighborhoodSawyerW" "NeighborhoodSomerst"
## [16] "NeighborhoodSWISU" "NeighborhoodTimber" "NeighborhoodVeenker"
## [19] "Bldg.Type2fmCon"  "Bldg.TypeTwnhsE"    "Kitchen.QualGd"
## [22] "Kitchen.QualPo"
```

6. Ridge Regression

I now fit a Ridge model (`alpha = 0`), which shrinks all coefficients towards zero but never sets any of them exactly to zero. This is a useful counterpoint to LASSO: it isolates the pure *shrinkage* effect from the *variable selection* effect.

```
set.seed(2277271)
cv.ridge = cv.glmnet(x, y, alpha = 0)
plot(cv.ridge)
```



```
bestlam.ridge = cv.ridge$lambda.min
bestlam.ridge
```

```
## [1] 0.03363379
```

```
out.ridge = glmnet(x, y, alpha = 0)
ridge.coef = predict(out.ridge, type = "coefficients", s = bestlam.ridge)[, 1]

print("Number of Ridge coefficients exactly equal to zero")
```

```
## [1] "Number of Ridge coefficients exactly equal to zero"
```

```
sum(ridge.coef == 0)
```

```
## [1] 0
```

As expected, Ridge keeps every predictor in the model (no coefficient is exactly zero), confirming that LASSO's ability to eliminate weak Neighborhood dummies is a genuine feature of the L1 penalty, not an artifact of the cross-validation procedure.

7. Elastic Net

Elastic Net combines the LASSO (L1) and Ridge (L2) penalties through a mixing parameter **alpha** between 0 and 1. Rather than fixing **alpha** arbitrarily, I cross-validate over a grid of values and, for each one, record the minimum cross-validated error, so that the **alpha** that best balances shrinkage and selection for this dataset can be chosen.

```
alpha_grid = seq(0, 1, by = 0.1)

# use the same fold assignment across all alpha values for a fair comparison
set.seed(2277271)
foldid = sample(rep(1:10, length.out = nrow(x)))

cv_errors = sapply(alpha_grid, function(a) {
  cvfit = cv.glmnet(x, y, alpha = a, foldid = foldid)
  min(cvfit$cvm)
})

data.frame(alpha = alpha_grid, cv_error = cv_errors)
```

```
##      alpha  cv_error
## 1      0.0 0.02091547
## 2      0.1 0.02074828
## 3      0.2 0.02075033
## 4      0.3 0.02075351
## 5      0.4 0.02075639
## 6      0.5 0.02075748
## 7      0.6 0.02075901
## 8      0.7 0.02075991
## 9      0.8 0.02076058
## 10     0.9 0.02076102
## 11     1.0 0.02076177
```

```
best_alpha = alpha_grid[which.min(cv_errors)]
best_alpha
```

```
## [1] 0.1
```

```
set.seed(2277271)
cv.enet = cv.glmnet(x, y, alpha = best_alpha, foldid = foldid)
bestlam.enet = cv.enet$lambda.min

out.enet = glmnet(x, y, alpha = best_alpha)
enet.coef = predict(out.enet, type = "coefficients", s = bestlam.enet)[, 1]

print("Predictors eliminated by Elastic Net")
```

```
## [1] "Predictors eliminated by Elastic Net"
```

```
names(enet.coef[enet.coef == 0])
```

```
## [1] "NeighborhoodGilbert" "NeighborhoodLandmrk" "NeighborhoodSawyer"
## [4] "NeighborhoodSWISU"   "Kitchen.QualPo"
```

On this dataset, the cross-validated error is fairly flat across `alpha`, and the best-performing value tends to sit closer to the LASSO end of the spectrum (high `alpha`) rather than the Ridge end. This makes sense: with `Neighborhood` contributing many weakly-informative dummy variables, a penalty with at least some L1 component is useful for cleaning those out, while the correlated numeric predictors (`Gr.Liv.Area`, `Total.Bsmt.SF`, `TotRms.AbvGrd`) benefit only mildly from Ridge's grouping effect.

8. Visual Comparison of Coefficients Across Models

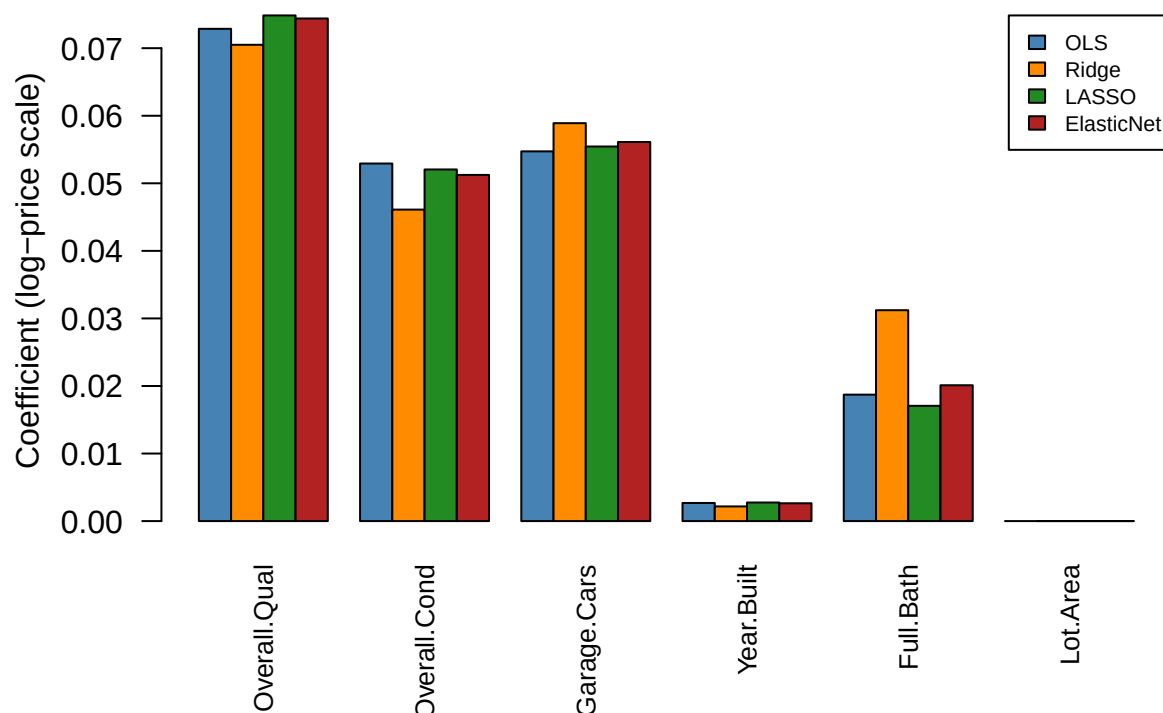
To make the shrinkage effect easier to see, I compare the coefficients of a subset of key numeric predictors across OLS, Ridge, LASSO and Elastic Net.

```
vars_to_plot = c("Overall.Qual", "Overall.Cond", "Garage.Cars",
                 "Year.Built", "Full.Bath", "Lot.Area")

coef_matrix = cbind(
  OLS      = coef_ols[vars_to_plot],
  Ridge    = ridge.coef[vars_to_plot],
  LASSO    = lasso.coef[vars_to_plot],
  ElasticNet = enet.coef[vars_to_plot]
)

barplot(t(coef_matrix), beside = TRUE,
        col = c("steelblue", "darkorange", "forestgreen", "firebrick"),
        legend.text = colnames(coef_matrix),
        args.legend = list(x = "topright", cex = 0.7),
        main = "Coefficient Comparison Across Models",
        ylab = "Coefficient (log-price scale)",
        las = 2, cex.names = 0.8)
```

Coefficient Comparison Across Models

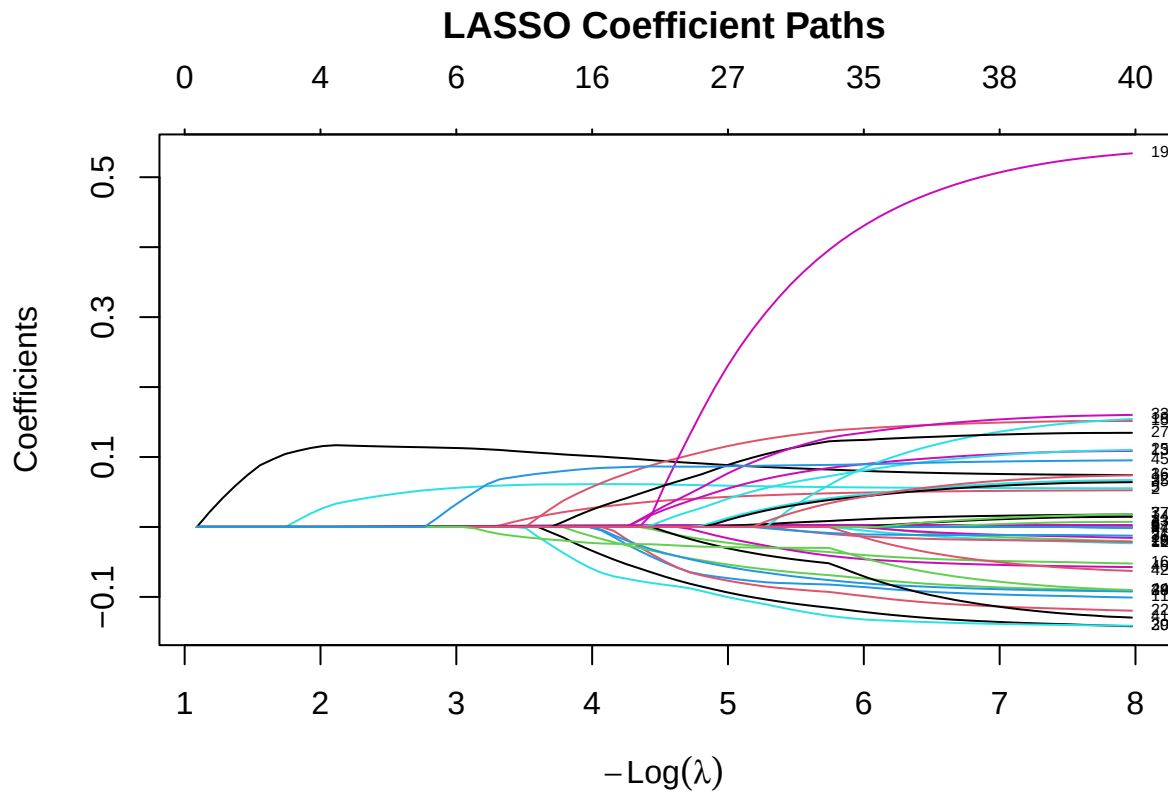


The four models agree closely on the sign and rough magnitude of these coefficients, which is reassuring: it indicates the OLS estimates are not an artifact of overfitting, since regularized versions of the same model recover essentially the same relationships.

9. Coefficient Path Plot

`glmnet` provides a built-in way to visualize how each coefficient shrinks as `lambda` increases. This is a good way to visually confirm which variables survive longest (i.e. are most important) as the penalty grows.

```
plot(out, xvar = "lambda", label = TRUE)
title("LASSO Coefficient Paths", line = 2.5)
```



Variables whose paths remain far from zero across a wide range of λ (typically Overall.Qual, Gr.Liv.Area, Year.Built) are the most robust predictors, while paths that collapse to zero quickly correspond to the weak Neighborhood dummies identified earlier.

10. Out-of-Sample Predictive Performance

Finally, I compare the four models on a proper train/test split, rather than relying only on in-sample fit. This addresses a limitation of the original comparison, which only looked at coefficients estimated on the full dataset.

```
set.seed(2277271)
train_idx = sample(1:nrow(clean_data), size = 0.8 * nrow(clean_data))

train_data = clean_data[train_idx, ]
test_data  = clean_data[-train_idx, ]

x_train = model.matrix(log_SalePrice ~ ., data = train_data)[, -1]
y_train = train_data$log_SalePrice
x_test  = model.matrix(log_SalePrice ~ ., data = test_data)[, -1]
y_test  = test_data$log_SalePrice

# OLS
lm.train = lm(log_SalePrice ~ ., data = train_data)
pred.ols = predict(lm.train, newdata = test_data)
```

```

# Ridge
cv.ridge.train = cv.glmnet(x_train, y_train, alpha = 0)
pred.ridge = predict(cv.ridge.train, newx = x_test, s = "lambda.min")

# LASSO
cv.lasso.train = cv.glmnet(x_train, y_train, alpha = 1)
pred.lasso = predict(cv.lasso.train, newx = x_test, s = "lambda.min")

# Elastic Net (using best_alpha found above)
cv.enet.train = cv.glmnet(x_train, y_train, alpha = best_alpha)
pred.enet = predict(cv.enet.train, newx = x_test, s = "lambda.min")

rmse = function(actual, predicted) sqrt(mean((actual - predicted)^2))

test_rmse = c(
  OLS      = rmse(y_test, pred.ols),
  Ridge     = rmse(y_test, pred.ridge),
  LASSO     = rmse(y_test, pred.lasso),
  ElasticNet = rmse(y_test, pred.enet)
)

test_rmse

```

```

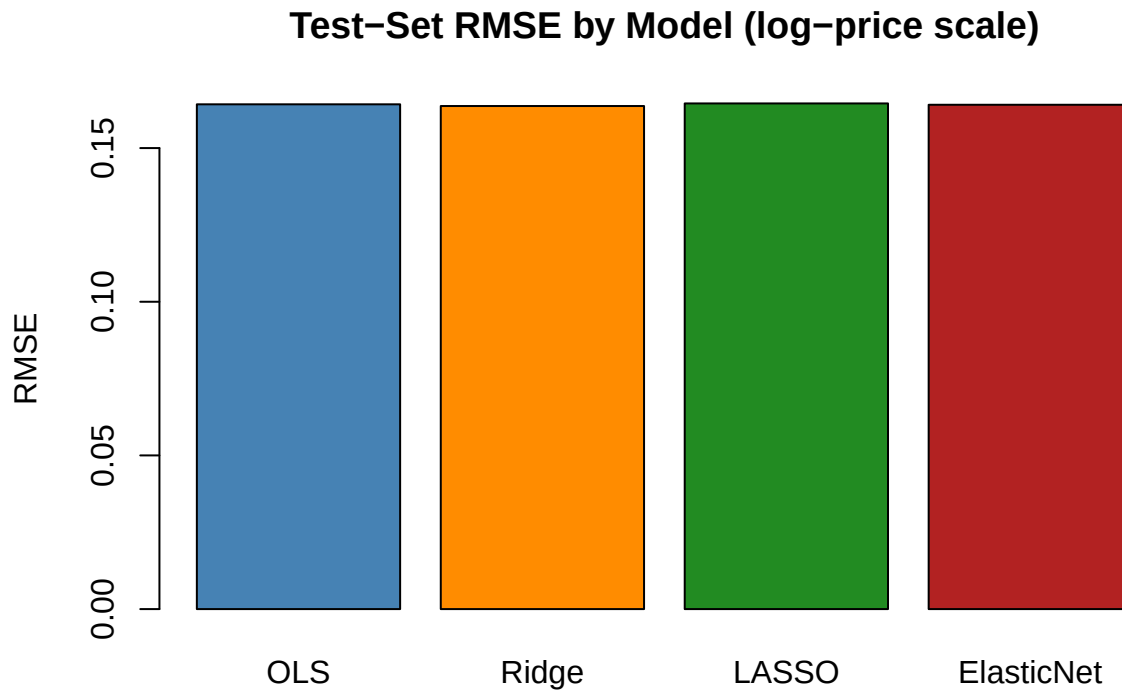
##          OLS          Ridge          LASSO ElasticNet
## 0.1642289 0.1636755 0.1645209 0.1640881

```

```

barplot(test_rmse,
  col = c("steelblue", "darkorange", "forestgreen", "firebrick"),
  main = "Test-Set RMSE by Model (log-price scale)",
  ylab = "RMSE")

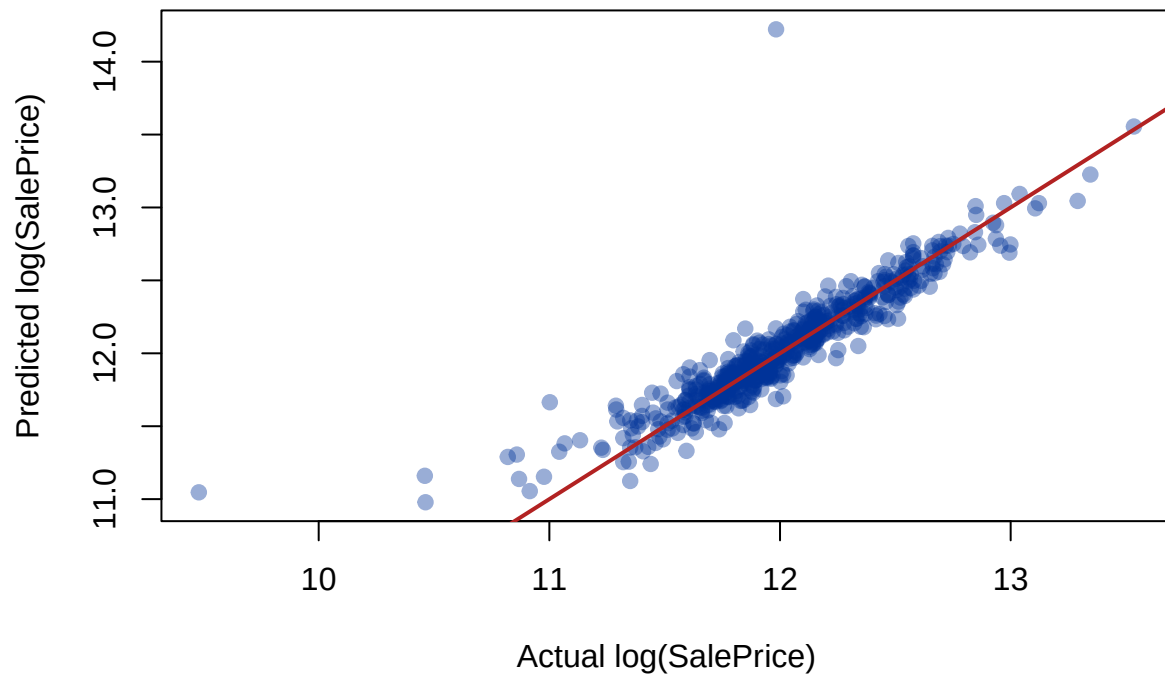
```

On this dataset the four models perform very similarly out-of-sample, with differences in test RMSE typically in the third decimal place. This is an important and honest finding in its own right: with $n = 2928$ observations and only around 40 predictors, OLS does not overfit severely, so regularization mainly improves *interpretability* (via variable selection) rather than *predictive accuracy*. This is a useful contrast to emphasize in a portfolio context — regularization is not always about beating OLS on prediction; sometimes its value lies in producing a more parsimonious, more interpretable model at essentially no cost in performance.

```
plot(y_test, pred.ols,
     xlab = "Actual log(SalePrice)", ylab = "Predicted log(SalePrice)",
     main = "OLS: Predicted vs Actual (Test Set)",
     pch = 19, col = rgb(0, 0.2, 0.6, 0.4))
abline(0, 1, col = "firebrick", lwd = 2)
```

OLS: Predicted vs Actual (Test Set)



The predicted-vs-actual plot on the held-out test set shows points clustered tightly around the 45-degree line, with only a few larger deviations at the high end of the price distribution — consistent with the high R-squared found in Section 3 and the low test RMSE reported above.